

SpaceCDF API Documentation

REST + WebSocket reference

SpaceCDF · v6

2026

Overview

SpaceCDF exposes a JSON REST API and a WebSocket channel for real-time concurrent-design collaboration. This document is the authoritative reference for engineers integrating SpaceCDF with external tools, building automation, or extending the platform.

Property	Value
Base URL (default)	<code>http://localhost:8000/api</code>
Content type	<code>application/json; charset=utf-8</code>
Authentication	Bearer token (Authorization header) — see §Auth
Total endpoints	105+
WebSocket path	<code>/ws/session/{session_id}</code>
OpenAPI spec	<code>/openapi.json</code>
Interactive docs	<code>/docs</code> (Swagger), <code>/redoc</code>

STANDARD REFERENCE

Standard reference SpaceCDF follows REST conventions consistent with the Microsoft API Guidelines — <https://github.com/microsoft/api-guidelines>. The OpenAPI specification is generated by FastAPI — <https://fastapi.tiangolo.com/>.

Authentication

All endpoints (except `/health` and `/openapi.json`) require a Bearer token:

```
GET /api/studies/abc123 HTTP/1.1
Host: cdf.example.org
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpX...
Content-Type: application/json
```

Scope	Permission
<code>read</code>	GET on studies, sessions, positions, KB
<code>write</code>	POST/PUT/PATCH on studies, sessions, parameters
<code>admin</code>	DELETE, parametric data edit, equipment DB import
<code>instructor</code>	Create cohort sessions, view all teams' studies

Tokens are issued by the auth provider configured in `packages/spacecdf-server/src/spacecdf_server/auth.py`. In classroom mode, instructors create tokens via the **Cohort** tab.

Conventions

HTTP STATUS CODES

Code	Meaning	Typical cause
200	OK	Successful GET / PATCH
201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Invalid request body or query params
401	Unauthorized	Missing or invalid token
403	Forbidden	Token valid but lacks required scope
404	Not Found	Resource ID unknown
409	Conflict	Stale state — re-run design before retry
422	Unprocessable Entity	Schema validation failed (Pydantic)
429	Too Many Requests	Rate limit hit
500	Internal Server Error	Bug — file a GitHub issue with <code>request_id</code>

PAGINATION

Collection endpoints accept `limit` and `cursor` query parameters and return a `next_cursor` field in the response:

```
{
  "items": [...],
  "next_cursor": "eyJpZCI6MTIzfQ"
}
```

Default limit 50, maximum 200.

RATE LIMITS

Class	Per-token quota	Per-IP quota
Read	600 / min	1200 / min
Write	120 / min	240 / min
Optimizer / sensitivity	10 / min	20 / min

429 responses include `Retry-After` (seconds) and `X-RateLimit-Reset` (epoch).

ERROR ENVELOPE

```
{
  "code": "E_BUD_022",
  "message": "Mass margin violated.",
  "details": {
    "subsystem": "payload",
    "actual_margin_pct": 4.1,
    "required_margin_pct": 10.0
  },
  "request_id": "req_01HF..."
}
```

Endpoint index

Verb	Path	Section
POST	/design/quick-design	Core Design
POST	/studies/	Core Design
GET	/studies/{id}	Core Design
POST	/lifecycle/orbit-trade	Lifecycle
POST	/lifecycle/class-advisor	Lifecycle
POST	/lifecycle/mission-trade	Lifecycle
GET	/lifecycle/spectrum/bands	Lifecycle
POST	/lifecycle/spectrum/itu-api-template	Lifecycle
POST	/lifecycle/spectrum/iaru-template	Lifecycle
POST	/lifecycle/regulatory/rssa	Lifecycle / Regulatory
POST	/lifecycle/regulatory/export-assessment	Lifecycle / Regulatory
POST	/lifecycle/regulatory/copuos-registration	Lifecycle / Regulatory
POST	/lifecycle/regulatory/eol-report	Lifecycle / Regulatory
GET	/lifecycle/parametric-data	Lifecycle
POST	/lifecycle/duty-cycles	Lifecycle
GET	/lifecycle/consistency/{study_id}	Lifecycle
POST	/lifecycle/trade-study	Lifecycle / Trade
GET	/lifecycle/trade-templates	Lifecycle / Trade
POST	/lifecycle/constellation/design	Lifecycle / Constellation
GET	/lifecycle/beyond-leo/orbits	Lifecycle / Beyond-LEO
POST	/lifecycle/beyond-leo/transfer	Lifecycle / Beyond-LEO
POST	/lifecycle/beyond-leo/dsn-link	Lifecycle / Beyond-LEO
GET	/lifecycle/requirements/generate/{study_id}	Requirements
POST	/lifecycle/requirements/validate	Requirements
POST	/lifecycle/requirements/check-compliance	Requirements
GET	/engineering/equipment/{domain}/search	Engineering
GET	/engineering/equipment/needs/{study_id}	Engineering
POST	/engineering/equipment/check-compatibility	Engineering
POST	/engineering/equipment/budget-impact	Engineering
GET	/engineering/verification	Engineering
GET	/engineering/cost	Engineering

Verb	Path	Section
POST	/engineering/analysis/sensitivity	Engineering
GET	/engineering/analysis/eol-curves	Engineering
POST	/engineering/analysis/trade-study	Engineering
POST	/engineering/impact-preview	Engineering
GET	/ecss/phases	ECSS
GET	/ecss/compliance/{phase_id}	ECSS
GET	/ecss/margins/{study_id}	ECSS
GET	/ecss/dids	ECSS
POST	/ecss/dids/{did_type}/generate	ECSS
GET	/optimize/config	Optimizer
POST	/optimize/sessions/{session_id}	Optimizer
GET	/optimize/runs/{run_id}	Optimizer
POST	/optimize/sensitivity/{session_id}	Optimizer
POST	/sessions/	Sessions
GET	/sessions/{id}	Sessions
GET	/sessions/{id}/history	Sessions
WS	/ws/session/{session_id}	Sessions
GET	/positions/	Positions
GET	/positions/{id}/guidance	Positions
POST	/positions/answers	Positions
GET	/positions/answers	Positions
GET	/kb/components/{category}	KB
GET	/kb/categories	KB
POST	/exports/docs/{format}	Exports
POST	/exports/mbse/{study_id}	Exports
POST	/exports/fsw/{study_id}	Exports

(Plus 30+ specialised endpoints — see `/openapi.json` for the authoritative list.)

Core design

POST /DESIGN/QUICK-DESIGN

Run the full design loop (20 agents, convergence in seconds).

Request body:

```
{
  "requirements": {
    "name": "EOSAT-1",
    "mission_type": "earth_observation",
    "spacecraft_class": "nano",
    "orbit": { "orbit_type": "sso", "altitude_km": 450, "inclination_deg":
98.0 },
    "payloads": [
      { "name": "Multispectral Imager", "mass_kg": 1.5, "power_w": 8,
"data_rate_mbps": 100 }
    ],
    "design_lifetime_years": 3
  }
}
```

Response (200):

```
{
  "converged": true,
  "iterations": 7,
  "total_time_s": 1.43,
  "parameters": {
    "battery_capacity_wh": { "value": 24.0, "unit": "Wh" },
    "solar_array_area_m2": { "value": 0.32, "unit": "m^2" },
    "antenna_gain_dbi": { "value": 4.0, "unit": "dBi" }
  },
  "budgets": { "mass_kg": 6.4, "power_w": 12.7, "link_margin_db": 5.6 },
  "warnings": [],
  "conflicts": []
}
```

POST /STUDIES/

Create a study. Persists requirements + mission need for later reference.

GET /STUDIES/{ID}

Retrieve study state. Returns the same shape as `quick-design` plus a `created_at` and `updated_at`.

Lifecycle services

POST /LIFECYCLE/ORBIT-TRADE

Compute orbit trade study with scored candidates.

Request:

```
{
  "target_gsd_m": 5.0,
  "target_revisit_days": 3,
  "target_latitude_band": "polar",
  "aperture_m": 0.15,
  "max_cost_meur": 50,
  "min_lifetime_years": 5,
  "mission_type": "earth_observation"
}
```

Response:

```
{
  "candidates": [
    {
      "name": "SSO 600 km – 10:30 LTAN",
      "altitude_km": 600,
      "inclination_deg": 97.79,
      "achievable_gsd_m": 4.3,
      "revisit_days": 3.1,
      "natural_lifetime_years": 12.0,
      "total_score": 0.91,
      "rank": 1
    }
  ]
}
```

POST /LIFECYCLE/CLASS-ADVISOR

Recommend spacecraft class from performance / programmatic targets.

POST /LIFECYCLE/MISSION-TRADE

Evaluate space vs non-space alternatives. Filtered by `mission_type`; includes constellation option when economics suggest.

GET /LIFECYCLE/SPECTRUM/BANDS

Available frequency bands filtered by mission type and licence type.

Query params: `mission_type`, `license_type` (`amateur` / `experimental` / `commercial`), `data_rate_mbps`.

POST /LIFECYCLE/SPECTRUM/ITU-API-TEMPLATE

Generate ITU Advance Publication Information filing template.

POST /LIFECYCLE/SPECTRUM/IARU-TEMPLATE

Generate IARU amateur coordination request template.

POST /LIFECYCLE/REGULATORY/RSSSA

Generate Canadian RSSSA filing template.

POST /LIFECYCLE/REGULATORY/EXPORT-ASSESSMENT

Generate export-control classification assessment.

POST /LIFECYCLE/REGULATORY/COPUOS-REGISTRATION

Generate UN Registration Convention Article IV template.

POST /LIFECYCLE/REGULATORY/EOL-REPORT

Generate end-of-life analysis report (consistent with IADC 25-yr rule).

GET /LIFECYCLE/PARAMETRIC-DATA

Return all parametric model data (mass fractions, cost fractions, power duty cycles, SA power tables) with source citations.

POST /LIFECYCLE/DUTY-CYCLES

Estimate power duty cycles for a mission configuration.

Request: { spacecraft_class, mission_type, comms_band, eclipse_fraction }.

GET /LIFECYCLE/CONSISTENCY/{STUDY_ID}

Run full design consistency check. Returns health score, issues by category.

POST /LIFECYCLE/TRADE-STUDY

Run a tabular trade study with criteria, weightings, and options.

```
{
  "name": "Comms band trade",
  "criteria": [
    { "id": "data_rate", "name": "Data rate", "weight": 4, "direction":
"max" },
    { "id": "license_complexity", "name": "Licence complexity", "weight": 2,
"direction": "min" }
  ],
  "options": [
    { "id": "uhf", "name": "UHF amateur", "scores": { "data_rate": 1, "li-
cense_complexity": 1 } },
    { "id": "s", "name": "S-band", "scores": { "data_rate": 4,
"license_complexity": 3 } }
  ]
}
```

GET /LIFECYCLE/TRADE-TEMPLATES

List pre-built trade-study templates (orbit, component, ground, architecture).

POST /LIFECYCLE/CONSTELLATION/DESIGN

Design Walker-delta constellation for coverage targets.

GET /LIFECYCLE/BEYOND-LEO/ORBITS

List beyond-LEO orbit options (MEO, GEO, HEO, lunar, interplanetary) with environment data.

POST /LIFECYCLE/BEYOND-LEO/TRANSFER

Compute transfer orbit Δv from LEO to target orbit.

POST /LIFECYCLE/BEYOND-LEO/DSN-LINK

Compute deep-space link budget for DSN communication.

GET /LIFECYCLE/REQUIREMENTS/GENERATE/{STUDY_ID}

Generate SMART requirements from study objectives and functions.

POST /LIFECYCLE/REQUIREMENTS/VALIDATE

Validate a requirement against SMART criteria.

POST /LIFECYCLE/REQUIREMENTS/CHECK-COMPLIANCE

Check non-compliance and get resolution options.

Engineering services

GET /ENGINEERING/EQUIPMENT/{DOMAIN}/SEARCH

Search KB for equipment compatible with the current design.

Domains: power , aocs , link , propulsion , structure , data , thermal , integration .

GET /ENGINEERING/EQUIPMENT/NEEDS/{STUDY_ID}

Determine which equipment categories are needed based on requirements.

POST /ENGINEERING/EQUIPMENT/CHECK-COMPATIBILITY

Check RF compatibility between transponder and antenna.

POST /ENGINEERING/EQUIPMENT/BUDGET-IMPACT

Compute live budget impact of current equipment selections.

GET /ENGINEERING/VERIFICATION

Run requirement verification against current design state.

GET /ENGINEERING/COST

Get cost estimate for current design.

POST /ENGINEERING/ANALYSIS/SENSITIVITY

Run parametric sensitivity sweep (Morris screening).

GET /ENGINEERING/ANALYSIS/EOL-CURVES

Get end-of-life degradation curves.

POST /ENGINEERING/ANALYSIS/TRADE-STUDY

Run equipment-level trade study.

POST /ENGINEERING/IMPACT-PREVIEW

Preview downstream impact of parameter changes (dry run — no execution).

ECSS compliance

GET /ECSS/PHASES

List available ECSS phases.

GET /ECSS/COMPLIANCE/{PHASE_ID}

Get expected DRDs and SpaceCDF coverage for a phase.

GET /ECSS/MARGINS/{STUDY_ID}

Check all budget margins against ECSS phase-appropriate policy.

GET /ECSS/DIDS

List available DID (Document Item Description) types.

POST /ECSS/DIDS/{DID_TYPE}/GENERATE

Generate an ECSS DID document. Types: `mrd`, `ts`, `ird`, `semp`, `rmp`, `conops`, `test_plan`.

Optimizer

GET /OPTIMIZE/CONFIG

Get available objectives and design variables (filtered by `mission_type` if provided).

Query params: `mission_type`, `has_propulsion`, `pointing_accuracy_deg`.

POST /OPTIMIZE/SESSIONS/{SESSION_ID}

Start an optimisation run (single-objective or Pareto NSGA-II).

GET /OPTIMIZE/RUNS/{RUN_ID}

Get optimisation run status and results.

POST /OPTIMIZE/SENSITIVITY/{SESSION_ID}

Run Morris-screening sensitivity analysis.

Sessions (concurrent design)

POST /SESSIONS/

Create a new design session.

GET /SESSIONS/{ID}

Get session state.

GET /SESSIONS/{ID}/HISTORY

Get edit history for a session.

WS /WS/SESSION/{SESSION_ID}

Real-time collaboration WebSocket. Message types:

Type	Direction	Payload
parameter_update	client → server	{ parameter, value, position }
state_update	server → all	full design state diff
convergence_complete	server → all	iterations, time, conflicts
participant_joined	server → all	display name + position
participant_left	server → all	display name
chat	client ↔ server	low-priority position chat

GET /POSITIONS/

List all engineering positions with roles and key questions.

GET /POSITIONS/{ID}/GUIDANCE

Get computed guidance for a position based on the current design.

POST /POSITIONS/ANSWERS

Save a position-question answer.

GET /POSITIONS/ANSWERS

Get all saved position answers.

Knowledge base

GET /KB/COMPONENTS/{CATEGORY}

Get all components in a category.

GET /KB/CATEGORIES

List all component categories.

Exports

POST /EXPORTS/DOCS/{FORMAT}

Generate design-review documents (SRR, PDR, CDR).

POST /EXPORTS/MBSE/{STUDY_ID}

Generate MBSE JSON export (ECSS-E-TM-10-25A style).

POST /EXPORTS/FSW/{STUDY_ID}

Generate FSW (flight-software) skeleton stub for the OBC.

Per-resource schemas (selected)

STUDY

Field	Type	Re- quired	Notes
id	string	✓	Server-assigned
name	string	✓	≤ 80 chars
mission_type	enum	✓	earth_observation · comms · science · tech_demo · pnt · defence
spacecraft_class	enum	✓	nano · micro · small · medium · large
orbit	Orbit	✓	See below
payloads	Pay- load[]	✓	At least one
design_lifetime_years	number	—	default 3
created_at	string (ISO 8601)	✓	UTC
updated_at	string (ISO 8601)	✓	UTC

ORBIT

Field	Type	Required	Notes
<code>orbit_type</code>	enum	✓	<code>leo</code> · <code>sso</code> · <code>meo</code> · <code>geo</code> · <code>heo</code> · <code>lunar</code>
<code>altitude_km</code>	number	✓	for <code>leo</code> / <code>sso</code>
<code>inclination_deg</code>	number	✓	0 – 180
<code>raan_deg</code>	number	—	default auto

PAYLOAD

Field	Type	Required	Notes
<code>name</code>	string	✓	
<code>mass_kg</code>	number	✓	
<code>power_w</code>	number	✓	average
<code>peak_power_w</code>	number	—	default = <code>power_w</code>
<code>data_rate_mbps</code>	number	✓	average
<code>aperture_m</code>	number	—	required for optical

(Other resource schemas: `Requirement`, `LinkBudget`, `TradeStudy`, `RiskItem`, `Equipment`, `OptimizerRun`, `SensitivityRun` — see `/openapi.json`.)

SDK compatibility

SDK	Version	Status
Python <code>spacecdf-client</code>	0.4.x	Stable
TypeScript <code>@spacecdf/client</code>	0.3.x	Beta
OpenAPI generators	—	Use <code>openapi.json</code> directly
Postman collection	—	Live at <code>/postman.json</code>

Python example:


```
from spacecdf_client import Client
c = Client("http://localhost:8000", token="...")
result = c.design.quick_design({
    "requirements": {
        "name": "Demo", "mission_type": "earth_observation",
        "spacecraft_class": "nano",
        "orbit": {"orbit_type": "sso", "altitude_km": 450, "inclination_deg":
98.0},
        "payloads": [{"name": "Imager", "mass_kg": 1.5, "power_w": 8,
"data_rate_mbps": 100}]
    }
})
print(result["budgets"])
```

Reference list

- **OpenAPI spec** — `/openapi.json` (auto-generated by FastAPI)
- **FastAPI docs** — <https://fastapi.tiangolo.com/>
- **Microsoft API Guidelines** — <https://github.com/microsoft/api-guidelines>
- **CCSDS PUS** — <https://public.ccsds.org/Pubs/660x0g3.pdf>
- **ECSS-E-TM-10-25A** (MBSE schema) — <https://ecss.nl/>
- **NASA Open MBEE** — <https://www.openmbee.org/>
- **SpaceCDF source** — <https://github.com/FNewland/SpaceCDF>